HDOC Day-69

Recursive Fibonacci Series and Recursive Function Call Count

Problem Statement

Write a C program using recursion to generate the first n Fibonacci numbers and count the number of recursive calls made. The series starts with 0 and 1. This solution counts every invocation of fibonacci(), including the initial invocation for each term.

Algorithm

1. Start.

2. Declare a global unsigned long long counter recursive_calls and initialize it to 0.

3. Define fibonacci(n): increment recursive_calls once on entry; return 0 if n=0; return 1 if n=1; otherwise return fibonacci(n-1)+fibonacci(n-2).

4. Read n and validate that it is an integer between 1 and 20. This classroom limit avoids excessive execution time for naive recursion.

5. For i from 0 to n-1, call fibonacci(i) and display the returned value separated by spaces.

6. Display the total number of recursive function calls.

7. Stop.

Test Case Table

Test case	Input n	Expected series / message	Call count
1	1	0	1
2	2	0 1	2
3	3	0 1 1	5
4	4	0 1 1 2	10
5	5	0 1 1 2 3	19
6	6	0 1 1 2 3 5	34
7	8	0 1 1 2 3 5 8 13	100
8	10	0 1 1 2 3 5 8 13 21 34	276
9	0	Invalid input	—
10	21	Invalid input	—

Note: For invalid input, the program displays: Invalid input. Enter an integer from 1 to 20.

C Program

```
#include <stdio.h>

unsigned long long recursive_calls = 0;
```

```
unsigned long long fibonacci(int n)
{
    recursive_calls++;
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main(void)
{
    int n, i;
    printf("Enter number of Fibonacci terms (1 to 20): ");
    if (scanf("%d", &n) != 1 || n < 1 || n > 20) {
        printf("Invalid input. Enter an integer from 1 to 20.\n");
        return 1;
    }
    printf("First %d Fibonacci numbers: ", n);
    for (i = 0; i < n; i++) {
        printf("%llu%s", fibonacci(i), i == n - 1 ? "\n" : " ");
    }
    printf("Total recursive function calls: %llu\n", recursive_calls);
    return 0;
}
```

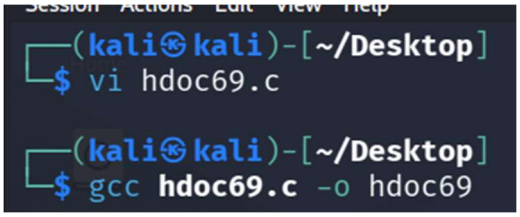
Compilation and Execution

In Kali Linux, save the program as hdoc69.c and run:

gcc hdoc69.c -o hdoc69

./hdoc69

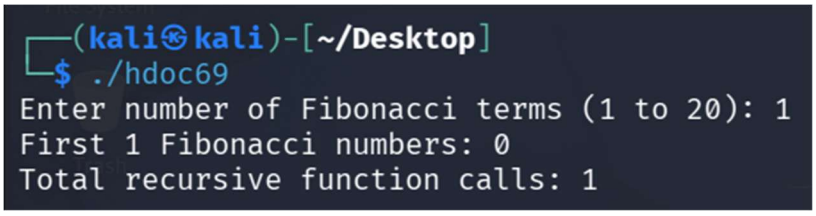
Compilation and Execution



Test Case 1

Input n: 1

Expected output: First 1 Fibonacci numbers: 0 | Total recursive function calls: 1



Test Case 2

Input n: 2

Expected output: First 2 Fibonacci numbers: 0 1 | Total recursive function calls: 2

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc69
Enter number of Fibonacci terms (1 to 20): 2
First 2 Fibonacci numbers: 0 1
Total recursive function calls: 2
```

Test Case 3

Input n: 3

Expected output: First 3 Fibonacci numbers: 0 1 1 | Total recursive function calls: 5

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc69
Enter number of Fibonacci terms (1 to 20): 3
First 3 Fibonacci numbers: 0 1 1
Total recursive function calls: 5
```

Test Case 4

Input n: 4

Expected output: First 4 Fibonacci numbers: 0 1 1 2 | Total recursive function calls: 10

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc69
Enter number of Fibonacci terms (1 to 20): 4
First 4 Fibonacci numbers: 0 1 1 2
Total recursive function calls: 10
```

Test Case 5

Input n: 5

Expected output: First 5 Fibonacci numbers: 0 1 1 2 3 | Total recursive function calls: 19

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc69
Enter number of Fibonacci terms (1 to 20): 5
First 5 Fibonacci numbers: 0 1 1 2 3
Total recursive function calls: 19
```

Test Case 6

Input n: 6

Expected output: First 6 Fibonacci numbers: 0 1 1 2 3 5 | Total recursive function calls: 34

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc69
Enter number of Fibonacci terms (1 to 20): 6
First 6 Fibonacci numbers: 0 1 1 2 3 5
Total recursive function calls: 34
```

Test Case 7

Input n: 8

Expected output: First 8 Fibonacci numbers: 0 1 1 2 3 5 8 13 | Total recursive function calls: 100

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc69
Enter number of Fibonacci terms (1 to 20): 8
First 8 Fibonacci numbers: 0 1 1 2 3 5 8 13
Total recursive function calls: 100
```

Test Case 8

Input n: 10

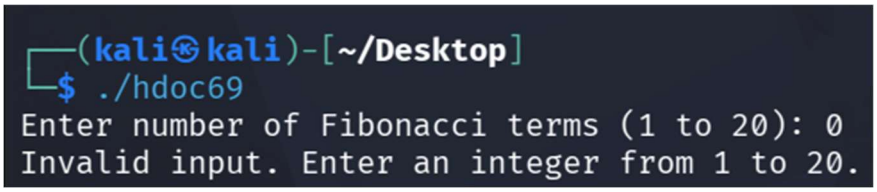
Expected output: First 10 Fibonacci numbers: 0 1 1 2 3 5 8 13 21 34 | Total recursive function calls: 276

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc69
Enter number of Fibonacci terms (1 to 20): 10
First 10 Fibonacci numbers: 0 1 1 2 3 5 8 13 21 34
Total recursive function calls: 276
```

Test Case 9

Input n: 0

Expected output: Invalid input. Enter an integer from 1 to 20.



Test Case 10

Input n: 21

Expected output: Invalid input. Enter an integer from 1 to 20.

